gRPC - Cheat Sheet

Contents

gRPC is Google's RPC framework. You define services in a `.proto` file, the compiler generates client and server code in multiple languages, and the runtime handles serialization, transport, and streaming over HTTP/2. Built for low-latency, polyglot service-to-service communication. Often a better fit than REST for internal microservices.

1. What gRPC Is

Three main pieces:

1. **Protocol Buffers (protobuf):** binary serialization format. Compact, fast, schema-driven.
2. **HTTP/2 transport:** multiplexed, streaming-capable.
3. **Generated stubs:** the compiler reads your `.proto` and outputs idiomatic client/server code in Java, Go, Python, C++, and more.

The result: you call a method on a stub, gRPC marshals the request, sends it over the wire, the server unmarshals, runs your handler, and the response travels back. From the caller's perspective it looks like a local method call.

2. gRPC vs REST

Aspect	gRPC	REST
Transport	HTTP/2	HTTP/1.1 (usually)
Payload	Protobuf (binary)	JSON (text)
Schema	Required (`.proto`)	Optional (OpenAPI)
Streaming	First-class (4 modes)	Limited (SSE, WebSocket)
Code gen	Built-in	Tool-dependent
Browser support	Needs gRPC-Web	Native
Discoverability	`grpcurl`, reflection	`curl`, browser
Best for	Service-to-service	Public APIs, browser clients

Pick gRPC for internal microservices where performance and type safety matter. Pick REST when browser clients, third-party developers, or curl-friendly debugging matter more.

3. Protocol Buffers Basics

Define messages and services in a `.proto` file.

```
syntax = "proto3";

package user.v1;

option java_package = "com.example.user.v1";
option java_multiple_files = true;

message User {
  int64 id = 1;
  string name = 2;
  string email = 3;
  repeated string roles = 4;
  UserStatus status = 5;
}

enum UserStatus {
  USER_STATUS_UNSPECIFIED = 0;
  USER_STATUS_ACTIVE = 1;
  USER_STATUS_DISABLED = 2;
}
```

Key rules:

- Every field has a **type, name, and tag number**. The tag is what the wire format uses. Never change a tag once it's published.

- Tags 19000-19999 are reserved for protobuf internals.
- Use `repeated` for arrays and lists.
- Enums always include `0` as the unspecified default.
- `proto3` (the modern version) drops `required`. Every field is optional by default.
- File and field names use `snake_case`. Messages and enums use `CamelCase`.

4. Service Definition

```
service UserService {
  rpc GetUser(GetUserRequest) returns (User);
  rpc ListUsers(ListUsersRequest) returns (ListUsersResponse);
  rpc CreateUser(CreateUserRequest) returns (User);
  rpc DeleteUser(DeleteUserRequest) returns (google.protobuf.Empty);
  rpc StreamUsers(StreamUsersRequest) returns (stream User);
  rpc ChatWithSupport(stream ChatMessage) returns (stream ChatMessage);
}

message GetUserRequest {
  int64 id = 1;
}
```

Each `rpc` line names a method, its request type, and its response type. The `stream` keyword marks streaming directions.

5. The Four RPC Types

Type	Request	Response	Example use
Unary	One	One	`GetUser(id)` returns a user
Server streaming	One	Many	`StreamLogs(filter)` pushes log lines over time
Client streaming	Many	One	`UploadFile(chunks)` returns a summary
Bidirectional streaming	Many	Many	Live chat, real-time sync

Syntax:

```
// Unary
rpc GetUser(GetUserRequest) returns (User);

// Server streaming
rpc StreamLogs(LogFilter) returns (stream LogEntry);

// Client streaming
rpc UploadFile(stream FileChunk) returns (UploadSummary);

// Bidirectional
rpc Chat(stream ChatMessage) returns (stream ChatMessage);
```

6. Generated Java Code

After running protoc (typically through a Gradle or Maven plugin), you get:

- Message classes (`User`, `GetUserRequest`, etc.) with builders.
- A service base class (`UserServiceGrpc.UserServiceImplBase`) to extend on the server.
- Stub classes for clients: `UserServiceBlockingStub`, `UserServiceFutureStub`, `UserServiceStub` (async).

Server side

```
public class UserServiceImpl extends UserServiceGrpc.UserServiceImplBase {

    @Override
    public void getUser(GetUserRequest req, StreamObserver<User> responseObserver) {
        User user = User.newBuilder()
            .setId(req.getId())
            .setName("alice")
            .setEmail("alice@example.com")
            .build();

        responseObserver.onNext(user);
        responseObserver.onCompleted();
    }
}

// Wire up the server
Server server = ServerBuilder.forPort(8080)
    .addService(new UserServiceImpl())
    .build()
    .start();

server.awaitTermination();
```

Client side

```java
// Channel: long-lived connection to the server
ManagedChannel channel = ManagedChannelBuilder
    .forAddress("localhost", 8080)
    .usePlaintext() // for local dev; use TLS in production
    .build();

// Blocking stub: synchronous calls
UserServiceGrpc.UserServiceBlockingStub stub =
        UserServiceGrpc.newBlockingStub(channel);

User user = stub.getUser(GetUserRequest.newBuilder().setId(42).build());
System.out.println(user.getName());

channel.shutdown();
```

Three stub flavors:

- **Blocking:** synchronous, for simple cases.
- **Future:** returns `ListenableFuture<T>` for async.
- **Async (StreamObserver):** for streaming and full async control.

7. Channels and Stubs

A `ManagedChannel` represents a connection to a gRPC server. It's expensive to create. Keep it alive for the lifetime of your client.

```java
ManagedChannel channel = ManagedChannelBuilder
    .forTarget("user-service:8080")
    .keepAliveTime(30, TimeUnit.SECONDS)
    .keepAliveTimeout(5, TimeUnit.SECONDS)
    .build();
```

Stubs are lightweight wrappers around channels. You can create them per call or hold them as fields.

```java
UserServiceBlockingStub stub = UserServiceGrpc.newBlockingStub(channel)
    .withDeadlineAfter(2, TimeUnit.SECONDS)
    .withInterceptors(new AuthInterceptor());
```

Stub methods like `withDeadlineAfter` return a new stub with that config applied. Stubs are immutable.

8. Metadata and Interceptors

Metadata is gRPC's equivalent of HTTP headers. Used for auth tokens, tracing IDs, and other request context.

```java
// Sending metadata from client
Metadata headers = new Metadata();
Metadata.Key<String> AUTH = Metadata.Key.of("authorization",
        Metadata.ASCII_STRING_MARSHALLER);
headers.put(AUTH, "Bearer eyJhbGciOi...");

stub = MetadataUtils.attachHeaders(stub, headers);
```

Interceptors let you run code on every call. Common uses: auth, logging, metrics, tracing.

```java
public class AuthInterceptor implements ClientInterceptor {

    @Override
    public <ReqT, RespT> ClientCall<ReqT, RespT> interceptCall(
            MethodDescriptor<ReqT, RespT> method,
            CallOptions callOptions,
            Channel next) {

        return new ForwardingClientCall.SimpleForwardingClientCall<>(
                next.newCall(method, callOptions)) {

            @Override
            public void start(Listener<RespT> responseListener, Metadata headers) {
                headers.put(AUTH, "Bearer " + getCurrentToken());
                super.start(responseListener, headers);
            }
        };
    }
}
```

9. Error Handling

gRPC has its own status codes, separate from HTTP codes.

Code	Meaning
`OK`	Success
`CANCELLED`	Client cancelled the call
`INVALID_ARGUMENT`	Bad request payload
`DEADLINE_EXCEEDED`	Call timed out
`NOT_FOUND`	Resource doesn’t exist
`ALREADY_EXISTS`	Resource already exists
`PERMISSION_DENIED`	Authenticated but not authorized
`UNAUTHENTICATED`	Missing or invalid credentials
`RESOURCE_EXHAUSTED`	Rate limit or quota hit

Code	Meaning
`FAILED_PRECONDITION`	System not in the required state
`ABORTED`	Concurrency conflict
`UNIMPLEMENTED`	Method doesn't exist
`INTERNAL`	Server-side bug
`UNAVAILABLE`	Server down or overloaded
`DATA_LOSS`	Unrecoverable data corruption

Throw a `StatusRuntimeException` from the server:

```
throw Status.NOT_FOUND
    .withDescription("user " + id + " not found")
    .asRuntimeException();
```

Catch it on the client:

```
try {
    User user = stub.getUser(request);
} catch (StatusRuntimeException e) {
    Status.Code code = e.getStatus().getCode();
    if (code == Status.Code.NOT_FOUND) {
        // handle gracefully
    }
}
```

10. Deadlines and Cancellation

Deadlines propagate across services. If service A calls service B with a 5-second deadline and B calls C, C inherits the remaining budget.

```
// Per-call deadline
User user = stub
    .withDeadlineAfter(2, TimeUnit.SECONDS)
    .getUser(request);
```

Without a deadline, calls can hang forever. Always set one for production code.

Server-side, check whether the call is still alive:

```
if (Context.current().isCancelled()) {
    return; // client gave up, stop doing work
}
```

11. Authentication

TLS

The default for production. Configure on both sides.

```
// Server with TLS
Server server = ServerBuilder.forPort(8443)
    .useTransportSecurity(certFile, keyFile)
    .addService(new UserServiceImpl())
    .build();

// Client with TLS
ManagedChannel channel = ManagedChannelBuilder
    .forAddress("api.example.com", 443)
    .useTransportSecurity()
    .build();
```

Token-based auth

Pass a token in metadata. Usually a JWT or service-to-service credential.

```
CallCredentials creds = new BearerTokenCredentials("eyJhbGci...");
stub = stub.withCallCredentials(creds);
```

mTLS

Mutual TLS: client and server both present certificates. Common in service-mesh setups like Istio, Linkerd, and Cloud Run.

12. When to Use gRPC

Good fit:

- Internal microservice communication.
- Polyglot environments where consistent client code matters.
- Low-latency, high-throughput pipelines.
- Streaming workloads (server-push, real-time updates).

Bad fit:

- Public APIs consumed by browsers or third parties.
- Simple CRUD where REST and JSON are easier to debug.
- Environments without proper HTTP/2 support (legacy proxies, some load balancers).

13. Common Gotchas

- **Don't change tag numbers.** Once published, tags are part of the contract. Adding new fields is fine; renumbering breaks every existing client.
- **Don't reuse deleted tag numbers.** Lock them with `reserved 4, 5;` or `reserved "old_field_name";` inside the message.
- **Default values are invisible.** A `string` field with an empty value and a missing field look identical on the wire. If "absent" matters, use `optional` (proto3) or wrap the value in a message.
- **Blocking stubs block the calling thread.** In reactive or virtual-thread code, prefer the future or async stubs.
- **Channels are expensive.** Don't create a new `ManagedChannel` per request. Reuse one across the lifetime of your client.
- **Set deadlines as your first defense.** Retries are dangerous. If you must retry, do it once and only for idempotent calls. Retry storms can take down upstream services.
- **gRPC doesn't run in browsers natively.** Use gRPC-Web (a proxy layer translating between browser HTTP/1.1 and real gRPC) or expose a REST gateway.
- **HTTP/2 issues hit hard.** Some load balancers don't speak HTTP/2 correctly. Test the full network path before assuming gRPC will work end-to-end.
- **Streaming needs backpressure thinking.** A slow consumer can pile up data in memory if you don't flow-control properly.

14. Quick Reference

Concept	Key fact
Transport	HTTP/2
Payload	Protocol Buffers (binary)
Schema	`.proto` file, compiled to many languages
RPC types	Unary, server streaming, client streaming, bidi
Channel	Long-lived connection, expensive to create
Stub	Lightweight wrapper around a channel
Metadata	gRPC's headers, used for auth and tracing
Status codes	Different from HTTP (`NOT_FOUND` , `UNAVAILABLE` , etc.)
Deadlines	Propagate across calls. Always set one
Interceptors	Cross-cutting logic (auth, logging, metrics)
Browser support	Via gRPC-Web proxy
Best for	Internal microservices, streaming, polyglot